TD-TP 7 : SQL DDL + DML - À propos de gestion de Transports

On reprend l'énoncé à propos d'une entreprise de transports possédant une flotte de camions en ajoutant la notion de distance et d'autres petites modifications (notamment au niveau du nommage). Les camions sont utilisés pour des transports effectués pour le compte de clients.

Le personnel est composé de personnes qui sont chacune, soit secrétaire soit chauffeur. Une personne est caractérisée par un identifiant, un nom, un prénom et un salaire. Une personne secrétaire est responsable des contrats d'un ensemble de clients (au moins un). Les chauffeurs sont affectés à la conduite de camions pour le transport des marchandises et on connaît pour chaque chauffeur le nombre de kilomètres effectués depuis son embauche par l'entreprise (*cette fois, il s'agit d'un attribut calculé par rapport aux distances parcourues avec ses contrats*). On prend pour hypothèse simplificatrice qu'à l'issue d'un transport, un chauffeur est immédiatement disponible pour un autre transport. Il peut exister des chauffeurs affectés à aucun transport, mais chaque secrétaire est responsable d'au moins un contrat. Un chauffeur ne peut pas effectuer plus d'un contrat à la même date.

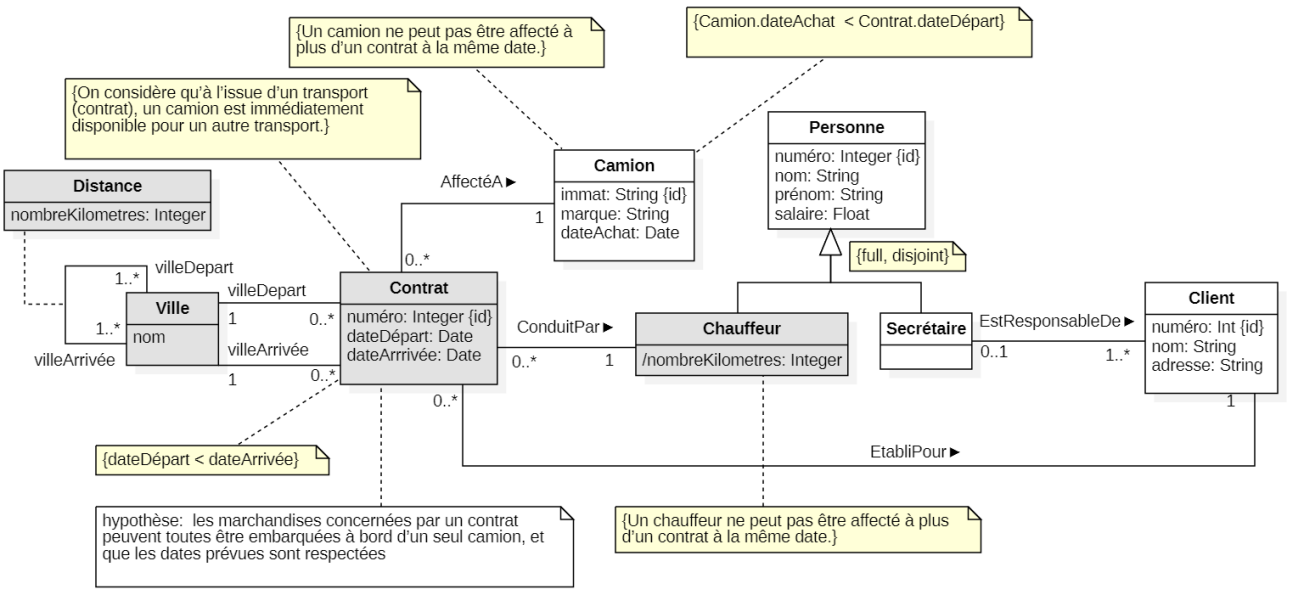
Les clients sont caractérisés chacun par un identifiant, un nom et une adresse. Tous les contrats d'un même client sont sous la responsabilité de la même personne, secrétaire de l'entreprise. Un contrat modélise le transport de marchandises d'une ville à une autre, avec une date de départ et une date d'arrivée prévues. On prend pour hypothèse que les marchandises concernées par un contrat peuvent toutes être embarquées à bord d'un seul camion, et que les dates prévues sont respectées. On garde dans la base de données des clients, même s'ils ne sont associés à aucun contrat.

Les camions sont chacun identifié par leur immatriculation. On modélise de plus la marque et la date d'achat de chaque camion. On considère qu'à l'issue d'un transport, un camion est immédiatement disponible pour un autre transport. Il peut exister des camions non encore affecté à un transport. Un camion ne peut pas être affecté à plus d'un contrat à la même date.

Les distances cette relation indique le nombre de kms entre une ville de départ et une ville d'arrivée.

1 Modèle conceptuel UML et modèle relationnel

Le nouveau modèle conceptuel UML (sans contraintes cette fois) :



Le schéma retenu pour la base de données est constitué des relations suivantes (les identifiants sont soulignés) :

Personnes (numero_personne, nom_personne, prénom_personne, salaire_personne)
/ <p, no, pr, s> ∈ Personnes ⟺ la personne identifiée par p a pour nom no, et prénom pr. Elle perçoit un salaire s. */*
Clients (numero_client, nom_client, adresse_client)
/ <c, n, a> ∈ Clients ⟺ le client identifié par c a pour nom n et réside à l'adresse a. */*
Camions (immat_camion, marque_camion, dateAchat_camion)
/ <i, m, d> ∈ Camions ⟺ le camion d'immatriculation i est de marque m. Il a été acheté par l'entreprise à la date d. */*
Contrats (numero_contrat, dateDepart_contrat, dateArrivee_contrat, villeDepart_contrat, villeArrivee, numero_client, immat_camion, numero_personne)
/ <t, dd, da, vd, va, c, i, p> ∈ Contrats ⟺ le contrat identifié par t est établi pour le compte du client c. Il s'agit du transport depuis la ville vd vers la ville va. Les dates de départ et d'arrivée prévues sont respectivement da et dd. Le camion affecté à ce contrat est celui d'immatriculation i, conduit par le chauffeur p. */*
Secrétaires (numero_personne, numero_client)
/ <p, c> ∈ Secrétaires ⟺ la personne secrétaire d'identifiant p est responsable des contrats du client d'identifiant c. */*
Chauffeurs_base (numero_personne)
/ <p> ∈ Chauffeurs_base ⟺ le chauffeur identifié par p */*
(view) Chauffeurs (numero_personne, nombreKilometresTotal_chauffeur)
/ <p, k> ∈ Chauffeurs ⟺ le chauffeur identifié par p a effectué k kms pour le compte de l'entreprise. */*
Distances (villeDepart_distance, villeArrivee_distance, nombreKilometres_distance)
/ <d, a, k> ∈ Distances ⟺ k est la distance en km entre la ville d et la ville a. <d, a> ∈ Distances ⟺ <a, d> ∈ Distances */*
Les domaines sont :
domaine (numero_personne) = domaine (numero_contrat) = domaine (numero_client) = entiers > 0
domaine (nom_personne) = domaine (prénom_personne) = domaine (immat_camion) = domaine (marque_camion) = chaînes de caractères
domaine (dateDepart_contrat) = domaine (dateArrivee_contrat) = domaine (dateAchat_camion) = date (à l'unité Jour)
domaine (adresse_client) = domaine (villeDepart_distance) = domaine (villeArrivee_distance) = {"Grenoble", "Paris", "Pau", etc.}
domaine (nombreKilometres_distance) = entiers >= 0, domaine (salaire_personne) = réels > 0

Les contraintes d'intégrité référentielle sont :

Personnes[numero_personne] = Secrétaires[numero_personne] ∪ Chauffeurs[numero_personne]
Secrétaires[numero_personne] ∩ Chauffeurs[numero_personne] = ∅
Secrétaires[numero_personne] ⊆ Personnes[numero_personne]
Chauffeurs[numero_personne] ⊆ Personnes[numero_personne]
Secrétaires[numero_client] ⊆ Clients[numero_client]
Contrats[numero_client] ⊆ Clients[numero_client]
Contrats[immat] ⊆ Camions[immat]
Contrats[numero_personne] ⊆ Chauffeurs[numero_personne]
Contrats[villeDepart_distance, villeArrivee_distance] ⊆ Distances[villeDepart_distance, villeArrivee_distance]

Les autres contraintes sont énumérées ci-dessous :

- La date de départ d'un transport est inférieure ou égale à la date d'arrivée.
- La date d'achat d'un camion est inférieure ou égale aux dates de départ des transports auxquels il est affecté.
- Chaque jour un camion est affecté à au plus un contrat.
- Chaque jour un chauffeur est affecté à au plus un contrat.
- Les villes de départ et arrivée sont différentes.
- Chaque secrétaire est responsable d'au moins un contrat.

2 Mise en place

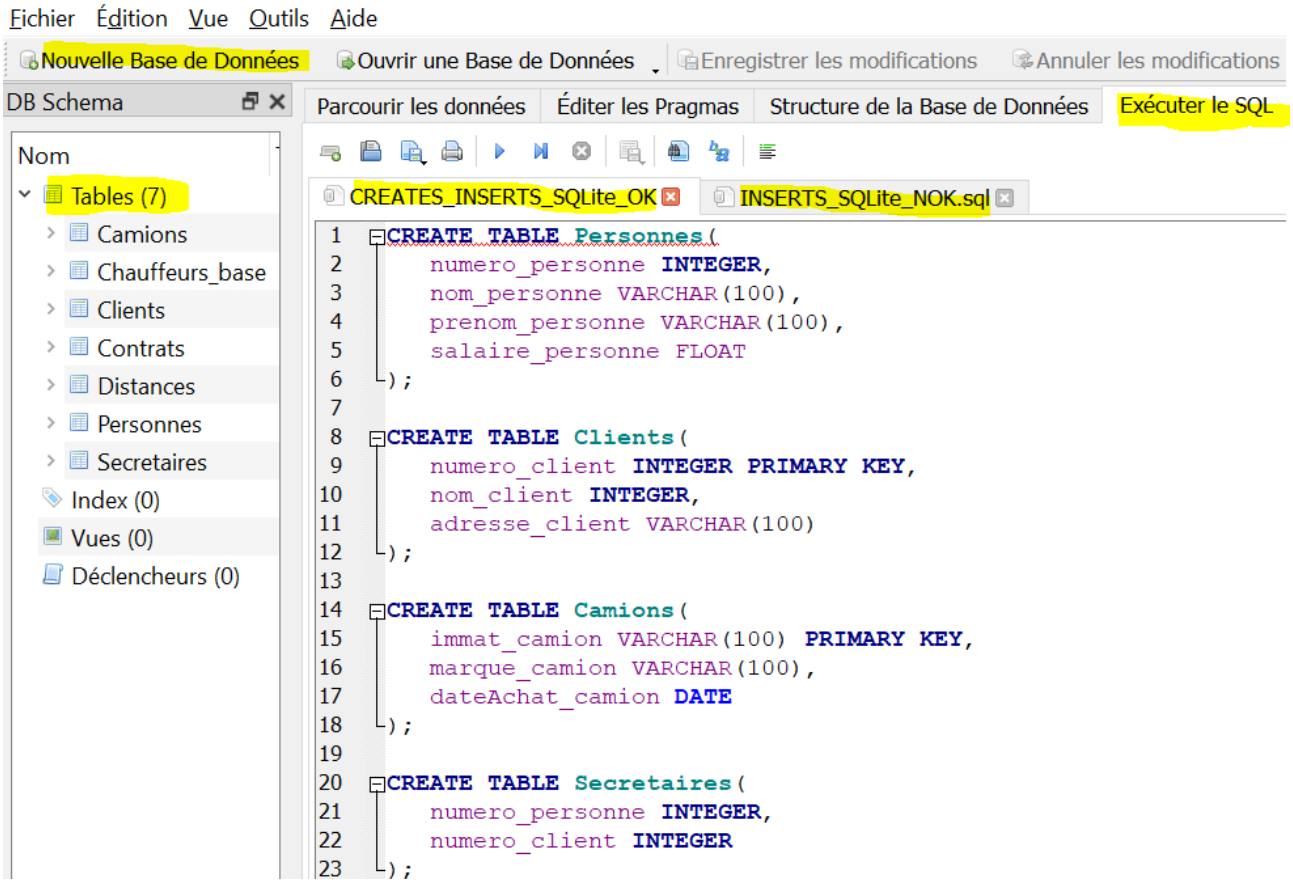
Pré-requis (déjà installés sur les machines de l'UFR IM2AG) :

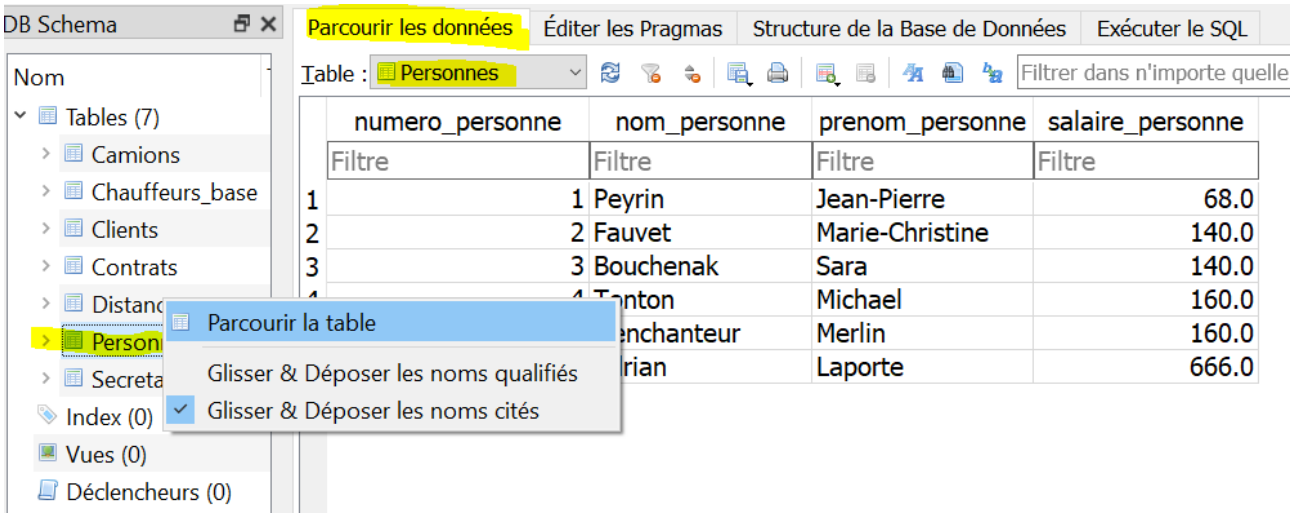
— SQLite3.

— SQLiteBrowser ou DBBrowser. Pour le lancer sur les machines de l'école, tapez : sqlitebrowser

Démarrage :

1. Créer une *Nouvelle Base de Données* sur SQliteBrowser.
2. Nommer la base de données. Par exemple : *CEBD_TDTP7_Transports.db*
3. Ouvrir **deux onglets** dans la partie *Exécuter le SQL*. Vous pouvez renommer les onglets comme dans l'image ci-dessous.
4. Ouvrir les deux scripts de l'énoncé `TDTP7.zip` chacun dans un onglet comme dans l'image ci-dessous.
5. Exécuter le script de création et insertion et vérifier que les données sont bien présentes dans la base de données.





3 Exercices

Question 1 :
A partir du schéma de relations, **compléter le script fourni de création de la base de données et d'insertion de données** (Script `TDTP7_CREATES_INSERTS_SQLite_OK.sql`) :

1. Ajouter les `DROP`s nécessaires pour pouvoir re-exécuter le script de manière aisée.
2. Implémenter les contraintes nécessaires pour éviter les "mauvaises" insertions du fichier (`Script TDTP7_INSERTS_SQLite_NOK.sql`). **Chaque ligne de ce fichier doit produire une erreur** levant l'exception d'une des contraintes d'intégrité. Aucune des lignes ne doit passer sans erreur. Penser à nommer correctement les contraintes afin d'expliciter les exceptions. **Attention** l'ajout de certaines contraintes peut interférer sur les `DROP`.

Question 2 :
Ajouter un mécanisme d'incrémentation automatique (autoincrement) pour les `Personnes` et les `Contrats`.

Question 3 :
Créer une vue (`VIEW`) `Chauffeurs` en fonction des distances parcourues selon leurs contrats. On veut calculer dynamiquement les kms des chauffeurs grâce à une vue.

Question 4 :
Proposer une requête `UPDATE` permettant d'augmenter de 10% les salaires des chauffeurs qui ont parcouru plus de 400km (modifier, avant, vos `INSERT`, si besoin, pour vérifier la requête).

Question 5 :
Utiliser l'option `ON CASCADE` permettant d'éliminer les chauffeurs ou secrétaires qui ne sont pas affectés à des contrats, si on les élimine de la table `Personnes` (ex. Peyrin)